First Steps in Verilog

Contemporary Digital Electronic Systems — Chapter 16

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. What Is Verilog?
3. Reading Verilog
4. Combinational Circuits
5. Sequential Circuits
6. Finite State Machines
7. Summary

Why This Chapter Matters

Every circuit in Chapters 4–15 could be drawn as a schematic. In practice designers write them in a hardware description language (HDL) and let a synthesis tool produce the gate-level netlist. This chapter is a *first exposure* to synthesizable Verilog: enough to describe, simulate and synthesise circuits you already know how to build.

After this chapter, you can:

1. Write a `module` with named inputs and outputs.
2. Describe combinational logic three ways (`assign`, `always @(*)` with `case`, gate primitives) and say which fits which situation.
3. Describe an edge-triggered register with `always @(posedge clk)`.
4. Write and read a three-process FSM.

Scope. A conservative, synthesizable subset of Verilog-2005 that mainstream tools accept: continuous assignments, `always` blocks, `case` statements. It is not a complete HDL reference.

© Muním Zabidi

An Opening Question

If Verilog “looks like C”, why does it compile to logic gates instead of CPU instructions?

What is Verilog really describing?

Hardware Is Concurrent

- Verilog is not a program whose statements run in order. Every `assign` and every `always` block exists *simultaneously* in the hardware, like a schematic where every component is always powered.
- Inside one `always` block, textual order implies an order of effect (last assignment wins), but there is no program counter.
- You define “what the hardware *is*”, not “what steps to execute”.

Syntactically valid $\neq$ synthesizable

A simulator accepts `initial` blocks, delays, floating-point literals, dynamic array sizes, non-constant loop bounds. No synthesis tool can build them. “My simulation passes” is not evidence the design fits a real FPGA: the synthesis report is the ground truth.

From Hardware Idea to Verilog

Hardware idea	Existing representation	Verilog
2-to-1 multiplexer	Truth table / equation / symbol	`assign y = s ? b : a;`
2-to-4 decoder	Truth table or case table	`always @(*)` with `case`
Full adder	Equations for sum and carry	`assign` or gate primitives
D flip-flop	Symbol / timing diagram	`always @(posedge clk)`
n-bit register	Symbol / RTL block	`always @(posedge clk)`, `reg [n-1:0]`
FSM	State diagram / table	Three processes: state register, next-state, output

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 Reading Verilog

4 Combinational Circuits

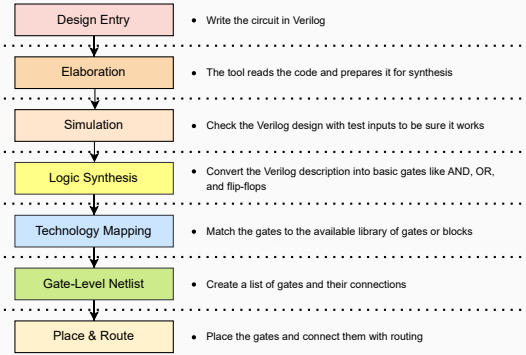
5 Sequential Circuits

6 Finite State Machines

7 Summary

Simulation and Synthesis

- An HDL describes circuits in text instead of schematics: easier to document, modify and scale.
- Simulation: check behaviour before building hardware.
- Synthesis: tools (Intel Quartus, Xilinx Vivado) map the code to gates for an FPGA or ASIC.
- FPGA: configurable logic blocks, lookup tables, routing. ASIC: standard cells, then place-and-route.



© Muním Zabidi

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 **Reading Verilog**

4 Combinational Circuits

5 Sequential Circuits

6 Finite State Machines

7 Summary

Reading Verilog: The Basics

- Modules: every circuit is a `module ... endmodule`; `input` and `output` declarations are the pins.
- Vectors: `[7:0]` is a bus of 8 wires numbered 7 down to 0.
- assign: a continuous, always-active connection: `assign y = a & b;`
- Gate primitives: `and`, `or`, `not`, `buf` instantiate gates like a schematic.

`3'b101`	3-bit binary, decimal 5
`8'hFF`	8-bit hex, decimal 255
`4'd9`	4-bit decimal 9

- Format: `<size>'<base><value>`.

© Munim Zabidi

Operators

Bitwise (per bit)	
`&`	AND
`|`	OR
`~`	NOT
`^`	XOR

Reduction (bus to 1 bit)	
`&d`	1 if all bits of `d` are 1
`|d`	1 if any bit is 1
`^d`	1 if an odd number of bits are 1

Other	
`?:`	conditional: 2-to-1 mux

`y = cond ? a : b;` means: if `cond` is 1, `y = a;` otherwise `y = b`.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 Reading Verilog

4 **Combinational Circuits**

5 Sequential Circuits

6 Finite State Machines

7 Summary

- Structural: instantiate and connect components (gate primitives or modules); mirrors hierarchical hardware.
- Gate-level: structural with primitive gates only; closest to a schematic. Best for learning.
- Dataflow: Boolean expressions with `assign`; concise, focuses on function.
- Behavioral: `always` blocks with `if` and `case`; the highest level, best for complex or sequential logic.

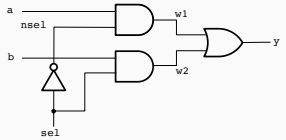
Learning path

Start gate-level to build intuition, move to dataflow for compact code, use structural for larger systems, and behavioral for complex or sequential functions.

© Munim Zabidi

2-to-1 Multiplexer

```
module mux2to1_gate (
  input a, b, sel,
  output y
);
  wire nsel, w1, w2;
  not (nsel, sel);
  and (w1, a, nsel);
  and (w2, b, sel);
  or (y, w1, w2);
endmodule
```



Gate-level: `not` makes the inverted select, two `and` gates enable the chosen input, `or` combines them.

```
module mux2to1_df (
  input a, b, sel,
  output y
);
  assign y = (sel) ? b : a;
endmodule
```

Dataflow: one conditional operator.

4-to-1 Multiplexer

```
module mux4to1_gate (
  input a, b, c, d,
  input [1:0] sel,
  output y
);
  wire n0, n1, w1, w2, w3, w4;
  not (n0, sel[0]);
  not (n1, sel[1]);
  and (w1, a, n1, n0);
  and (w2, b, n1, sel[0]);
  and (w3, c, sel[1], n0);
  and (w4, d, sel[1], sel[0]);
  or (y, w1, w2, w3, w4);
endmodule
```

```
module mux4to1_df (
  input a, b, c, d,
  input [1:0] sel,
  output y
);
  assign y = (sel == 2'b00) ? a :
             (sel == 2'b01) ? b :
             (sel == 2'b10) ? c : d;
endmodule
```

Gate-level: inverted selects, four `and` gates, one `or`.
Dataflow: nested conditional operators.

2-to-4 Decoder

```
module dec2to4_gate (
  input [1:0] a,
  output [3:0] y
);
  wire na0, na1;
  not (na0, a[0]);
  not (na1, a[1]);
  and (y[0], na1, na0); // 00
  and (y[1], na1, a[0]); // 01
  and (y[2], a[1], na0); // 10
  and (y[3], a[1], a[0]); // 11
endmodule
```

```
module dec2to4_df (
  input [1:0] a,
  output [3:0] y
);
  assign y = 4'b0001 << a;
endmodule
```

Gate-level: inverted inputs and four `and` gates give one-hot outputs. Dataflow: shift a one-hot value left by the input index.

Half Adder and Full Adder

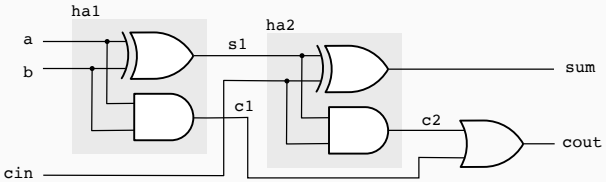
```
module half_adder (
  input a, b,
  output sum, carry
);
  xor (sum, a, b);
  and (carry, a, b);
endmodule
```

```
module half_adder_df (
  input a, b,
  output sum, carry
);
  assign sum = a ^ b;
  assign carry = a & b;
endmodule
```

```
module full_adder_gate (
  input a, b, cin,
  output sum, cout
);
  wire s1, c1, c2;
  half_adder ha1 (a, b, s1, c1);
  half_adder ha2 (s1, cin, sum, c2);
  or (cout, c1, c2);
endmodule
```

```
module full_adder_df (
  input a, b, cin,
  output sum, cout
);
  assign {cout, sum} = a + b + cin;
endmodule
```

Structural Full Adder



- Built from two half-adder modules and an OR gate.
- `ha1` adds `a` and `b`; `ha2` adds that sum and `cin`; the OR combines the two carries.
- Internal wires `s1`, `c1`, `c2` carry the intermediate signals.
- The dataflow version uses concatenation: `{cout, sum} = a + b + cin`.

© Muním Zabidi

2-Bit Comparator

```
module comp2bit_gate (
  input [1:0] a, b,
  output eq, gt, lt
);
  wire eq1, eq0, gt1, gt0, lt1, lt0;
  xnor (eq1, a[1], b[1]);
  xnor (eq0, a[0], b[0]);
  and (eq, eq1, eq0);
  and (gt1, a[1], ~b[1]);
  and (gt0, eq1, a[0], ~b[0]);
  or (gt, gt1, gt0);
  and (lt1, ~a[1], b[1]);
  and (lt0, eq1, ~a[0], b[0]);
  or (lt, lt1, lt0);
endmodule
```

```
module comp2bit_df (
  input [1:0] a, b,
  output eq, gt, lt
);
  assign eq = (a == b);
  assign gt = (a > b);
  assign lt = (a < b);
endmodule
```

Gate-level: `eq` when both bits match; `gt` when the first mismatched bit of `a` is larger; `lt` likewise for `b`. Dataflow: comparison operators state the intent directly.

8-to-3 Priority Encoder

Gate-level: OR gates form the code directly.

```
module prienc8to3_gate (
  input [7:0] d,
  output [2:0] y,
  output valid
);
  or (valid, d[0], d[1], d[2], d[3],
          d[4], d[5], d[6], d[7]);
  or (y[2], d[7], d[6], d[5], d[4]);
  or (y[1], d[7], d[6], d[3], d[2]);
  or (y[0], d[7], d[5], d[3], d[1]);
endmodule
```

Behavioral: priority is the order of the `if` conditions.

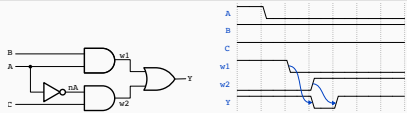
```
module prienc8to3_beh (
  input [7:0] d,
  output reg [2:0] y,
  output valid
);
  always @(*) begin
    if (d[7]) y = 3'b111;
    else if (d[6]) y = 3'b110;
    else if (d[5]) y = 3'b101;
    else if (d[4]) y = 3'b100;
    else if (d[3]) y = 3'b011;
    else if (d[2]) y = 3'b010;
    else if (d[1]) y = 3'b001;
    else y = 3'b000;
  end
  assign valid = |d;
endmodule
```

To change which input has priority, reorder the conditions instead of re-deriving the Boolean equations.

Hazards and Buffers

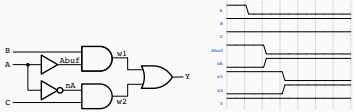
$Y = A \cdot B + \overline{A} \cdot C$ is functionally correct, but unequal gate delays can glitch Y when A changes (a static-1 hazard). The reliable fix is the consensus term: $Y = AB + \overline{A}C + BC$ holds Y high while the two terms swap roles.

```
module hazard_demo (
  input A, B, C,
  output Y
);
  wire nA, w1, w2;
  not (nA, A);
  and (w1, A, B);
  and (w2, nA, C);
  or (Y, w1, w2);
endmodule
```



A weaker approach: a `buf` delays A in the first product term, balancing arrival times with $\overline{A}$:

```
module hazard_free_demo (
  input A, B, C,
  output Y
);
  wire nA, Abuf, w1, w2;
  not (nA, A);
  buf (Abuf, A);
  and (w1, Abuf, B);
  and (w2, nA, C);
  or (Y, w1, w2);
endmodule
```



Delay balancing reduces the likelihood of a glitch. It is an illustration, not a dependable fix: the delay depends on the technology, and synthesis may remove the buffer.

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 Reading Verilog

4 Combinational Circuits

5 **Sequential Circuits**

6 Finite State Machines

7 Summary

- Question to keep in mind: which values are stored, and what event makes them update?
- Sequential logic stores state in flip-flops or latches. It is written in behavioral Verilog: `always @(posedge clk)` (or `negedge`) describes clocked updates; `if` and `case` give reset, enable and transitions.
- Naming: lowercase (`d`, `q`, `clk`) for single bits; uppercase or wider (`Q`, `DATA[7:0]`) for buses.

Simulation $\neq$ arbitrary software

Delays (#), dynamic arrays, floating-point literals and `$display` are accepted by simulators but cannot be synthesised. The synthesis report decides.

© Muním Zabidi

SR Latch and D Latch

Level-sensitive, responds immediately to `S` and `R`:

```
module sr_latch (
  input S, R,
  output reg Q
);
  always @(*) begin
    if (S && !R) Q = 1; // Set
    else if (!S && R) Q = 0; // Reset
    // S=R=0: hold; S=R=1: invalid
  end
endmodule
```

Transparent when `en` is high, otherwise holds:

```
module d_latch (
  input D, en, reset,
  output reg Q
);
  always @(*) begin
    if (reset) Q = 0;
    else if (en) Q = D; // transparent
    // else hold value
  end
endmodule
```

D, T and JK Flip-Flops

D: samples `D` on the rising edge.

```
module d_ff (
  input D, clk, reset,
  output reg Q
);
  always @(posedge clk
        or posedge reset)
    if (reset) Q <= 0;
    else Q <= D;
endmodule
```

T: toggles when `T = 1`.

```
module t_ff (
  input T, clk, reset,
  output reg Q
);
  always @(posedge clk
        or posedge reset)
    if (reset) Q <= 0;
    else if (T) Q <= ~Q;
endmodule
```

JK: set, reset, toggle.

```
module jk_ff (
  input J, K, clk, reset,
  output reg Q
);
  always @(posedge clk
        or posedge reset)
    if (reset) Q <= 0;
    else if (J && !K) Q <= 1;
    else if (!J && K) Q <= 0;
    else if (J && K) Q <= ~Q;
endmodule
```

Asynchronous reset: `posedge reset` is in the sensitivity list. Use non-blocking assignment (<=) for every register update.

4-Bit Counter and Shift Register

Up-counter with asynchronous reset:

```
module counter4 (
  input clk, reset,
  output reg [3:0] Q
);
  always @(posedge clk
         or posedge reset)
    if (reset) Q <= 4'b0000;
    else Q <= Q + 1;
endmodule
```

Shifts right, inserting `D` at the left:

```
module shiftreg4 (
  input clk, reset, D,
  output reg [3:0] Q
);
  always @(posedge clk
         or posedge reset)
    if (reset) Q <= 4'b0000;
    else Q <= {D, Q[3:1]};
endmodule
```

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 Reading Verilog

4 Combinational Circuits

5 Sequential Circuits

6 **Finite State Machines**

7 Summary

Level-to-Pulse Converter in Verilog

- The Moore level-to-pulse FSM of Chapter 13: one single-cycle pulse per rising edge of `L`, however long `L` stays high.
- Three states:
  - `ZERO`: waiting for `L` to rise; `P = 0`.
  - `EDGE`: one cycle after the rising edge; `P = 1`.
  - `ONE`: waiting for `L` to fall; `P = 0`.
- From `EDGE`: to `ZERO` if `L = 0`, else to `ONE`: exactly one pulse per rising edge.

Three-process structure

State register (clocked), next-state logic (combinational, a `case`), output logic (combinational). Separating them gives a clear, synthesizable, maintainable design.

The Three-Process FSM

```
module edge_detector (
  input clk, reset,
  input L, // input level signal
  output P // pulse output
);
  localparam [1:0] ZERO = 2'b00, EDGE = 2'b01, ONE = 2'b10;
  reg [1:0] state, next_state;

  // PART 1: State register
  always @(posedge clk or posedge reset)
    if (reset) state <= ZERO;
    else state <= next_state;

  // PART 2: Next-state logic
  always @(*) begin
    case (state)
      ZERO: next_state = L ? EDGE : ZERO;
      EDGE: next_state = L ? ONE : ZERO;
      ONE: next_state = L ? ONE : ZERO;
      default: next_state = ZERO;
    endcase
  end

  // PART 3: Output logic (Moore: depends only on state)
  assign P = (state == EDGE);
endmodule
```

`default` gives a recovery path: two bits can encode four values but only three are used.

Back to the Opening Question

Why does “C-like” Verilog compile to gates?

Because Verilog describes structure and behaviour that exist concurrently, not a sequence of instructions. `assign` makes combinational logic, `always @(*)` makes combinational logic from `if` or `case`, and `always @(posedge clk)` makes flip-flops. The synthesis tool turns those descriptions into gates and registers; there is no CPU to execute them.

Which values are stored, and what updates them?

What is stored is whatever is assigned in the `always` block and declared `reg`; the event is named in the sensitivity list. `always @(posedge clk)`: a rising edge, so a flip-flop. `always @(*)`: no event; it re-evaluates when any input changes. The latch examples infer latches deliberately, despite the warning in the Summary.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 What Is Verilog?

3 Reading Verilog

4 Combinational Circuits

5 Sequential Circuits

6 Finite State Machines

7 **Summary**

Summary

- A Verilog file describes hardware that exists concurrently, not a program run top to bottom.
- Combinational: `assign` for simple expressions; `always @(*)` for `case`/`if` logic. Every path must assign every output, or synthesis infers unintended latches.
- Sequential: `always @(posedge clk)` with non-blocking assignment (`<=`) for every register update; keep reset style consistent.
- FSMs are cleanest as three processes: state register, next-state logic, output logic. Add a `default` to every `case`.
- Simulation checks behaviour; synthesis checks implementability. A passing simulation does not guarantee a working chip.

Looking ahead

This is a first step. A full HDL course covers parameterised modules, timing closure, clock-domain crossings and SystemVerilog. Next: take a small design from Chapters 9–15, write it in Verilog, simulate it and synthesise it onto an inexpensive FPGA board.